

Teaching Exam

Subject – Computer Science

Topic – Database Query using SQL



Lecture No.- 1

By- Rahul Sir

Topics to be Covered

Topic



- 1:- SQL QUERY AND COMMAND ✓
- 2:- SUBQUERY ✓
- 3:- TYPES OF SQL QUERY ✓
- 4:- EXPLAIN DDL,DML,DQL,DCL,TCL ✓
- 5:- LOGICAL OPERATOR IN MYSQL ✓
- 6:- DISTINCT KEYWORD ✓



Topic : SQL

SQL, which means “Structured Query Language,” is the primary language used for managing and manipulating relational databases. We use SQL commands to tell the database what actions we want it to perform.

What are SQL commands?

SQL commands are the keywords that tell the database what kind of action to perform, like CREATE, SELECT, or INSERT. These commands perform various database operations, such as creating tables, inserting data, querying information, and controlling access and security.

SQL Statement: It is a complete instruction you give to the database. It is made up of one or more commands and other elements like table names, columns, values, or conditions. For example, “CREATE TABLE Customers” is a statement.

A SQL Query is a type of SQL statement that asks the database to return data. It always starts with the SELECT command.



Topic : Types of SQL Query

SQL commands are divided into five main groups based on their use. Knowing these groups helps you understand what different commands are meant to do, There five types of SQL command:-

Type	Full Form	What it does	Example
DDL	Data Definition Language	Makes or changes tables / Table Structure	Create, Alter, DROP
DML	Data Manipulation Language	Adds, changes, and deletes data	Insert, update, Delete, Truncate
DQL	Data Query Language	Asks questions to get data	Select
DCL	Data Control Language	Controls who can access data	Grant, Revoke
TCL	Transaction Control Language	Manages changes as one group	Commit, RollBack



Topic : Sub-Query

What is SQL Subquery?

In SQL, a subquery can be defined as a query **embedded** within another query. It is often used in the WHERE, HAVING, or FROM clauses of a statement. Subqueries are commonly used with **SELECT**, **UPDATE**, **INSERT**, and **DELETE** statements to achieve **complex filtering** and **data manipulation**. They are an essential tool when we need to perform operations like:

- **Filtering:** Getting specific records based on conditions derived from another query.
- **Aggregating:** Performing aggregate functions like SUM, COUNT, or AVG based on subquery results.
- **Updating:** Dynamically updating records based on values from other tables.
- **Deleting:** Deleting records from one table using criteria based on another.

While there is no universal syntax for subqueries, they are commonly used in **SELECT** statements as follows. This general syntax allows the outer query to use the results of the inner subquery for filtering or other operations.



Topic : Sub-Query

Common SQL Clauses for Subqueries

Subqueries are frequently used in specific **SQL clauses** to achieve more complex results. Here are the common clauses where subqueries are used:

- 1. WHERE Clause:** Subqueries in the WHERE clause help filter data based on the results of another query. For example, you can filter records based on values returned by a subquery.
- 2. FROM Clause:** Subqueries can be used in the **FROM** clause to treat the result of the subquery as a derived table or temporary table that can be joined with other tables.
- 3. HAVING Clause:** Subqueries in the **HAVING** clause allow you to filter aggregated data after performing group operations.

```
SELECT column_name  
FROM table_name  
WHERE column_name expression operator  
      (SELECT column_name FROM table_name WHERE ...);
```



Topic : Sub-Query

1. DATABASE TABLE

NAME	ROLL_NO	LOCATION	PHONE_NUMBER
Ram	101	Chennai	9988775566
Raj	102	Coimbatore	8877665544
Sasi	103	Madurai	7766553344
Ravi	104	Salem	8989898989
Sumathi	105	Kanchipuram	8989856868

Database Table

2. STUDENT TABLE

NAME	ROLL_NO	SECTION
Ravi	104	A
Sumathi	105	B
Raj	102	A



Topic : Sub-Query

Example 1: Fetching Data Using Subquery in WHERE Clause

This example demonstrates how to use a subquery to retrieve roll numbers of students in **section 'A'**, and the outer query uses those roll numbers to fetch corresponding details (name, location, and phone number) from the **DATABASE** table. This enables filtering based on results from another table.

Query:

```
SELECT NAME, LOCATION, PHONE_NUMBER  
FROM DATABASE  
WHERE ROLL_NO IN (  
    SELECT ROLL_NO FROM STUDENT WHERE SECTION 'A'  
)
```

IN

Output

NAME	LOCATION	PHONE_NUMBER
Ravi	Salem	8989898989
Rai	Coimbatore	8877665544





Topic : Create and Use Database in MySQL

The SQL CREATE DATABASE Statement:-

```
CREATE DATABASE databasename;
```

```
USE databasename;
```

EXAMPLE:-

```
CREATE DATABASE testDB;
```

```
USE testDB;
```

Create Database db1;

Create Database db2; *name*

Use db2; *name*



Topic : DDL Commands – Building the Structure

DDL stands for Data Definition Language. These commands are used to build and manage the structure of the database. They set up where your data will be stored

CREATE:-

int

1. CREATE: This command is used to create a new database object. For example, creating a new table, a view, or a database.

```
1 CREATE TABLE Employees (  
2   EmployeeID INT PRIMARY KEY,  
3   → FirstName VARCHAR(50),  
4   LastName VARCHAR(50),  
5   ...  
6 )
```

Roll in
Attribute
name

int

Data Definition Language (DDL)

CREATE

```
CREATE TABLE table_name ( column1  
datatype constraints, column2  
datatype constraints, ... );
```

Creates a new table, view, or database object

ALTER

```
ALTER TABLE table_name ADD  
column_name datatype;
```

Modifies an existing database object

DROP

```
DROP TABLE table_name;
```

Deletes an existing table, view, or other database object

TRUNCATE

```
TRUNCATE TABLE table_name;
```

Removes all records from a table, but not the table itself



Topic : DDL Commands – Building the Structure

DDL stands for Data Definition Language. These commands are used to build and manage the structure of the database. They set up where your data will be stored

ALTER:-

2. ALTER: This command is used to modify an existing database object, such as adding, deleting, or modifying columns in an existing table.

E_Id	EName	Emp
------	-------	-----

EmpId	EName Lastname	Mobile
-------	------------------------------	-------------------

Email

1 ALTER TABLE Employees

2 ~~ADD Email VARCHAR(100);~~

3 ~~DROP COLUMN Mobile;~~

4 MODIFY COLUMN LastName VARCHAR(150);

Data Definition Language (DDL)

CREATE

```
CREATE TABLE table_name ( column1  
datatype constraints, column2  
datatype constraints, ... );
```

Creates a new table, view, or database object

ALTER

```
ALTER TABLE table_name ADD  
column_name datatype;
```

Modifies an existing database object

DROP

```
DROP TABLE table_name;
```

Deletes an existing table, view, or other database object

TRUNCATE

```
TRUNCATE TABLE table_name;
```

Removes all records from a table, but not the table itself



Topic : DDL Commands – Building the Structure

DDL stands for Data Definition Language. These commands are used to build and manage the structure of the database. They set up where your data will be stored

DROP:-

3. DROP: This command is used to delete an entire database object, like a table, column, view, index, or even a database itself.

```
1  -- 1. Drop a column from the Employees table
2  ALTER TABLE Employees
3  DROP COLUMN Email;
4
5  -- 2. Drop the Employees table entirely
6  DROP TABLE Employees;
7
8  -- 3. Drop the database (for example, named CompanyDB)
9  DROP DATABASE CompanyDB;
```

Alter Table Employees Drop columns Email, Mobilen;

Data Definition Language (DDL)

CREATE

```
CREATE TABLE table_name ( column1
datatype constraints, column2
datatype constraints, ... );
```

Creates a new table, view, or database object

ALTER

```
ALTER TABLE table_name ADD
column_name datatype;
```

Modifies an existing database object

DROP

```
DROP TABLE table_name;
```

Deletes an existing table, view, or other database object

TRUNCATE

```
TRUNCATE TABLE table_name;
```

Removes all records from a table, but not the table itself



Topic : DDL Commands – Building the Structure



TEACHING
WALLAH

DDL stands for Data Definition Language. These commands are used to build and manage the structure of the database. They set up where your data will be stored

TRUNCATE:-

4. TRUNCATE: This command is used to delete all records from a table, without deleting the table. It resets the table to empty, but keeps its structure (columns, data types, constraints). It's a fast way to clear large data from a table.

Data Definition Language (DDL)

CREATE

```
CREATE TABLE table_name ( column1  
datatype constraints, column2  
datatype constraints, ... );
```

Creates a new table, view, or database object

ALTER

```
ALTER TABLE table_name ADD  
column_name datatype;
```

Modifies an existing database object

DROP

```
DROP TABLE table_name;
```

Deletes an existing table, view, or other database object

TRUNCATE

```
TRUNCATE TABLE table_name;
```

Removes all records from a table, but not the table itself

1 | **TRUNCATE TABLE** Employees;

→ name



Topic : DDL Commands – Building the Structure



TEACHING
WALLAH

DDL stands for Data Definition Language. These commands are used to build and manage the structure of the database. They set up where your data will be stored

RENAME TABLE AND RENAME COLUMN:-

5. RENAME: Used to change the name of a database object (like a table or column).

```
1  -- Rename the table
2  RENAME TABLE Employees TO Staff;
3
4  -- Rename a column (MySQL)
5  ALTER TABLE Staff old name Staff new name Name;
6  CHANGE COLUMN FullName Name VARCHAR(100);
```



Topic : DML Commands – Adding and Changing Data

DML stands for Data Manipulation Language. These commands work with the actual data stored inside the database structure you created with DDL. We use them to add, change, or remove data rows.

INSERT :-

Key DML Commands:

Employees

First Name	Last Name	Department
Rahul	Gupta	faculty
Sohan	Mawry	HR

1. INSERT: Used to add new rows or records into a table.

tablename
INSERT INTO Employees (First Name, Last Name, Department)

VALUES ('Rahul', 'Gupta', 'HR');

*Insert into Employees (First Name, Last Name, Department)
values ('Sohan', 'Mawrya', 'HR');*

Data Manipulation Language (DML)

SELECT

SELECT column1, column2, ... FROM table_name WHERE condition;

Retrieves data from a database

INSERT

INSERT INTO table_name (column1, column2, ...) VALUES (value1, value2, ...);

Adds new rows of data to a table

UPDATE

UPDATE table_name SET column1 = value1, column2 = value2, ... WHERE condition;

Modifies existing data in a table

DELETE

DELETE FROM table_name WHERE condition;

Removes rows from a table



Topic : DML Commands – Adding and Changing Data

DML stands for Data Manipulation Language. These commands work with the actual data stored inside the database structure you created with DDL. We use them to add, change, or remove data rows.

UPDATE :-

2. **UPDATE:** Used to change existing records in a table.

1 | UPDATE Employees SET Salary = Salary * 1.1 WHERE
Department = 'Engineering';

Handwritten annotations:
- "table name" points to Employees
- "Attribute name" points to Salary
- "10% increase" points to Salary * 1.1

Handwritten note:
Update Employees Set Salary = Salary * 1.15 Where Department = 'HR';

Data Manipulation Language (DML)

SELECT

SELECT column1, column2, ... FROM
table_name WHERE condition;
Retrieves data from a database.

INSERT

INSERT INTO table_name (column1,
column2, ...) VALUES (value1, value2,
...);
Adds new rows of data to a table.

UPDATE

UPDATE table_name SET column1 =
value1, column2 = value2, ... WHERE
condition;
Modifies existing data in a table.

DELETE

DELETE FROM table_name WHERE
condition;
Removes rows from a table.



Topic : DML Commands – Adding and Changing Data

DML stands for Data Manipulation Language. These commands work with the actual data stored inside the database structure you created with DDL. We use them to add, change, or remove data rows.

DELETE :-

3. DELETE: Used to remove records from a table.

ETA	Name	Department
1	Ram	HR
2	Shyam	Finance
3	Harishan	IT

1 DELETE FROM Employees WHERE Department = 'Finance';

Delete

and Where Eid=2;

Data Manipulation Language (DML)

SELECT

```
SELECT column1, column2, ... FROM  
table_name WHERE condition;
```

Retrieves data from a database

INSERT

```
INSERT INTO table_name (column1,  
column2, ...) VALUES (value1, value2,  
...);
```

Adds new rows of data to a table

UPDATE

```
UPDATE table_name SET column1 =  
value1, column2 = value2, ... WHERE  
condition;
```

Modifies existing data in a table

DELETE

```
DELETE FROM table_name WHERE  
condition;
```

Removes rows from a table



Topic : DQL Commands – Getting Information (These are the “Queries”)



TEACHING
WALLAH

DQL stands for Data Query Language. This is the command type you will use most often. Its only job is to retrieve data out of the database based on rules you give it. The central DQL command is **SELECT**. When people talk about “SQL Queries”, they usually mean using SELECT.

Query
1 | SELECT column1, column2, ... FROM table_name WHERE condition;

Sub Query *table name* *Sub Query*

*



Topic : DQL Commands – Getting Information (These are the “Queries”)

1 | **SELECT** column1, column2, ... **FROM** table_name **WHERE** condition;

How SELECT works:

You tell the database exactly what data you want to see and which table it should look in.

Simple SELECT examples:

- `SELECT * FROM Customers;` – Get all the data from the Customers table.
- `SELECT FirstName, LastName FROM Customers;` – Get only the first and last names from the Customers table.
- `SELECT * FROM Customers WHERE City = 'New York';` – Get all data for customers who live in New York.

When you run a SELECT command, the database gives you back the information you asked for.



Topic : DCL Commands – Who Can Do What

DCL stands for Data Control Language. These commands are used to manage access and permissions in a database. They control who can see, modify, or manage the data and database objects.

GRANT:-

Key DCL Commands:

1. **GRANT**: Use this command to give specific users certain permissions, like giving a user permission to read or modify data in a table.

```
1 | GRANT SELECT, INSERT ON Employees TO HR_Manager;
```

This grants the "HR_Manager" role the privileges to select and insert data into the "Employees" table.

Data Control Language (DCL)

GRANT ✓

GRANT permission ON object TO user;

Gives specific privileges to users

REVOKE ✓

REVOKE permission ON object FROM user;

Removes specific privileges from users

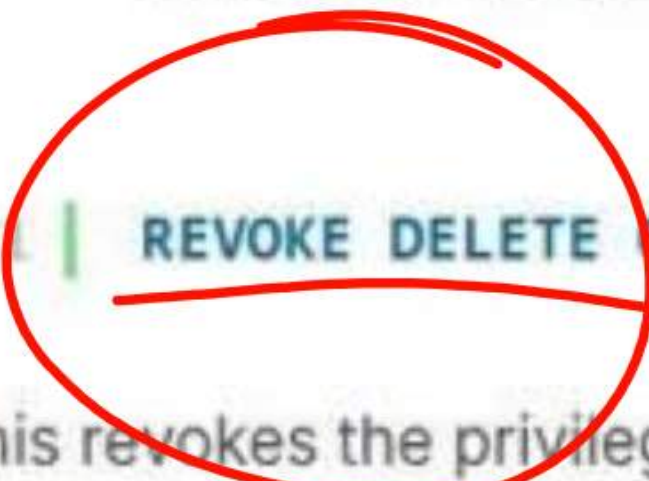


Topic : DCL Commands – Who Can Do What

DCL stands for Data Control Language. These commands are used to manage access and permissions in a database. They control who can see, modify, or manage the data and database objects.

REVOKE:-

2. **REVOKE:** This command is used to remove previously granted permissions from users. You can REVOKE their authorization if you don't want the user to access specific data.


REVOKE DELETE ON Customers FROM Sales_Team;

This revokes the privilege to delete data from the "Customers" table from the "Sales_Team" role.

Data Control Language (DCL)

GRANT

GRANT permission ON object TO user;
Gives specific privileges to users

REVOKE

REVOKE permission ON object FROM user;
Removes specific privileges from users



Topic : TCL Commands – Making Changes Safely

TCL stands for Transaction Control Language. These commands help manage transactions. A transaction is a set of SQL actions that should all happen together as a single unit. TCL ensures that if something goes wrong during a set of changes, the database can go back to how it was before.

BEGIN and COMMIT Command:-

Key TCL Commands:

1. BEGIN: Starts a transaction. It marks the beginning of a series of operations that should be treated as one unit.

```
1 BEGIN;  
2 -- SQL statements  
3 COMMIT;
```

2. COMMIT: Used to save all the changes you made during a transaction permanently.

```
1 BEGIN;  
2 -- SQL statements  
3 COMMIT;
```

This example begins a transaction, performs SQL statements, and then commits the changes to the database.

Transaction Control Language (TCL)

COMMIT ✓

```
COMMIT;
```

Saves all transactions to the database

ROLLBACK ✓

```
ROLLBACK;
```

Reverts to the state before the transactions

SAVEPOINT ✓

```
SAVEPOINT savepoint_name;
```

Creates points within transactions to ROLLBACK to



Topic : TCL Commands – Making Changes Safely

TCL stands for Transaction Control Language. These commands help manage transactions. A transaction is a set of SQL actions that should all happen together as a single unit. TCL ensures that if something goes wrong during a set of changes, the database can go back to how it was before.

ROLLBACK Command:-

3. ROLLBACK: Used to undo changes you made during a transaction if something went wrong.

```
1 | BEGIN;  
2 | -- SQL statements  
3 | ROLLBACK;
```

This example begins a transaction, performs SQL statements, and then rolls back the changes, restoring the database to its previous state.

Transaction Control Language (TCL)

COMMIT

```
COMMIT;
```

Saves all transactions to the database

ROLLBACK

```
ROLLBACK;
```

Reverts to the state before the transactions

SAVEPOINT

```
SAVEPOINT savepoint_name;
```

Creates points within transactions to ROLLBACK to



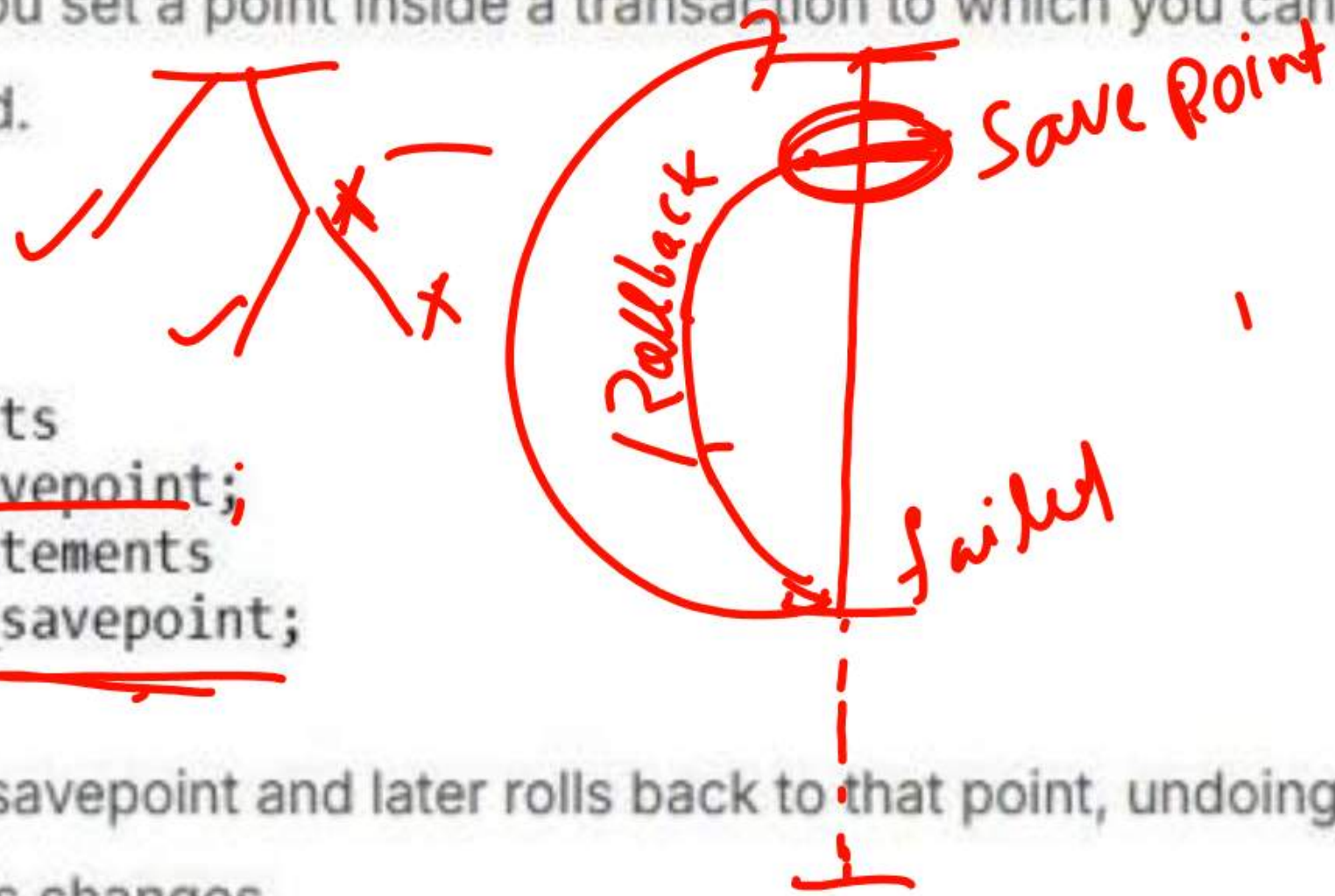
Topic : TCL Commands – Making Changes Safely

TCL stands for Transaction Control Language. These commands help manage transactions. A transaction is a set of SQL actions that should all happen together as a single unit. TCL ensures that if something goes wrong during a set of changes, the database can go back to how it was before.

SAVEPOINT Command:-

4. **SAVEPOINT**: It lets you set a point inside a transaction to which you can roll back if needed.

```
1 BEGIN;  
2 -- SQL statements  
3 SAVEPOINT my_savepoint;  
4 -- More SQL statements  
5 ROLLBACK TO my_savepoint;
```



This example creates a savepoint and later rolls back to that point, undoing some of the transaction's changes.

Transaction Control Language (TCL)

COMMIT

COMMIT;

Saves all transactions to the database

ROLLBACK

ROLLBACK;

Reverts to the state before the transactions

SAVEPOINT

SAVEPOINT savepoint_name;

Creates points within transactions to ROLLBACK to



Topic : Logical Operators in MySQL

The logical operators in MySQL are fundamental components of SQL statements that allow you to filter data from a table by combining numerous conditions in a WHERE clause. Logical operators are used in the creation of complicated queries that retrieve data based on specified parameters. MySQL has three logical operators: AND, OR, and NOT.

Truth Tables for OR, AND, and NOT Operators

X	Y	X OR Y	X AND Y	NOT X
true	true	true	true	false
true	false	true	false	false
false	true	true	false	true
false	false	false	false	true



Topic : Logical AND Operators in MySQL

name	age	department	salary
A	25	IT	60000
B	29	Sales	60000
C	35	Sales	45000
D	26	IT	52000

**SELECT * FROM employees
WHERE department = 'Sales'
AND salary > 50000;**

Output:

name	age	department	salary
B	29	Sales	60000



Topic : Logical OR Operators in MySQL

name	age	department	salary
A	25	IT	60000
B	29	Sales	60000
C	35	Sales	45000
D	26	IT	52000

```
SELECT * FROM employees  
WHERE department = 'Sales'  
OR department =  
'Marketing';
```

Output:

name	age	department	salary
A	25	Marketing	60000
B	29	Sales	60000



Topic : Logical NOT Operators in MySQL

name	age	department	salary
A	25	IT	60000
B	29	Sales	60000
C	35	Sales	45000
D	26	IT	52000

```
SELECT * FROM employees  
WHERE NOT department =  
'Sales';
```

Output:

name	age	department	salary
A	25	Marketing	60000
D	26	IT	52000



Topic :Explanation of Operator Precedence in MySQL



TEACHING
WALLAH

The order in which logical operators in MySQL are evaluated in a SQL query is called logical operator precedence in MySQL. **The precedence of logical operators is: NOT > AND > OR.** The higher precedence operators are evaluated first, followed by lower precedence operators. Let us understand how operator precedence works in MySQL with the help of our sample employees table.

name	age	department	salary
A	25	Marketing	60000
B	29	Sales	60000
C	35	Sales	45000
D	26	IT	52000

```
SELECT * FROM employees  
WHERE department = 'Sales' AND salary >  
50000 OR department = 'Marketing';
```

Output:

name	age	department	salary
B	29	Sales	60000
A	25	Marketing	60000



What is the SQL DISTINCT Keyword?

The distinct keyword is used in conjunction with the select keyword. It is helpful when there is a need to avoid duplicate values present in any specific columns/table. When we use distinct keywords only the **unique values** are fetched.

SYNTAX:-

```
SELECT DISTINCT column1, column2  
FROM table_name;
```

NOTE: If a distinct keyword is used with multiple columns, the distinct combination is displayed in the result set.



Topic :SQL DISTINCT Keyword



TEACHING
WALLAH

```
CREATE TABLE students (  
    ROLL_NO INT,  
    NAME VARCHAR(50),  
    ADDRESS VARCHAR(100),  
    PHONE VARCHAR(20),  
    AGE INT  
);  
  
INSERT INTO students (ROLL_NO, NAME, ADDRESS, PHONE, AGE)  
VALUES  
    (1, 'Shubham Kumar', '123 Main Street, Bangalore', '9876543210', 23),  
    (2, 'Shreya Gupta', '456 Park Road, Mumbai', '9876543211', 23),  
    (3, 'Naveen Singh', '789 Market Lane, Delhi', '9876543212', 26),  
    (4, 'Aman Chopra', '246 Forest Avenue, Kolkata', '9876543213', 22),  
    (5, 'Aditya Patel', '7898 Ocean Drive, Chennai', '9876543214', 27),  
    (6, 'Avdeep Desai', '34 River View, Hyderabad', '9876543215', 24),  
    (7, 'Shubham Kumar', '123 Main Street, Bangalore', '9876543210', 23), --  
Duplicate  
    (8, 'Shreya Gupta', '456 Park Road, Mumbai', '9876543211', 23), --  
Duplicate  
    (9, 'Naveen Singh', '789 Market Lane, Delhi', '9876543212', 26), --  
Duplicate  
    (10, 'Aditya Patel', '7898 Ocean Drive, Chennai', '9876543214', 27), --  
Duplicate  
    (11, 'Aman Chopra', '246 Forest Avenue, Kolkata', '9876543213', 22), --  
Duplicate  
    (12, 'Avdeep Desai', '34 River View, Hyderabad', '9876543215', 24); --  
Duplicate
```



Topic :SQL DISTINCT Keyword



TEACHING
WALLAH

```
CREATE TABLE students (  
    ROLL_NO INT,  
    NAME VARCHAR(50),  
    ADDRESS VARCHAR(100),  
    PHONE VARCHAR(20),  
    AGE INT  
);  
  
INSERT INTO students (ROLL_NO, NAME, ADDRESS, PHONE, AGE)  
VALUES  
    (1, 'Shubham Kumar', '123 Main Street, Bangalore', '9876543210', 23),  
    (2, 'Shreya Gupta', '456 Park Road, Mumbai', '9876543211', 23),  
    (3, 'Naveen Singh', '789 Market Lane, Delhi', '9876543212', 26),  
    (4, 'Aman Chopra', '246 Forest Avenue, Kolkata', '9876543213', 22),  
    (5, 'Aditya Patel', '7898 Ocean Drive, Chennai', '9876543214', 27),  
    (6, 'Avdeep Desai', '34 River View, Hyderabad', '9876543215', 24),  
    (7, 'Shubham Kumar', '123 Main Street, Bangalore', '9876543210', 23), --  
Duplicate  
    (8, 'Shreya Gupta', '456 Park Road, Mumbai', '9876543211', 23), --  
Duplicate  
    (9, 'Naveen Singh', '789 Market Lane, Delhi', '9876543212', 26), --  
Duplicate  
    (10, 'Aditya Patel', '7898 Ocean Drive, Chennai', '9876543214', 27), --  
Duplicate  
    (11, 'Aman Chopra', '246 Forest Avenue, Kolkata', '9876543213', 22), --  
Duplicate  
    (12, 'Avdeep Desai', '34 River View, Hyderabad', '9876543215', 24); --  
Duplicate
```



Topic :SQL DISTINCT Keyword



TEACHING
WALLAH

Output:

ROLL_NO	NAME	ADDRESS	PHONE	AGE
1	Shubham Kumar	123 Main Street, Bangalore	9876543210	23
2	Shreya Gupta	456 Park Road, Mumbai	9876543211	23
3	Naveen Singh	789 Market Lane, Delhi	9876543212	26
4	Aman Chopra	246 Forest Avenue, Kolkata	9876543213	22
5	Aditya Patel	7898 Ocean Drive, Chennai	9876543214	27
6	Avdeep Desai	34 River View, Hyderabad	9876543215	24
7	Shubham Kumar	123 Main Street, Bangalore	9876543210	23
8	Shreya Gupta	456 Park Road, Mumbai	9876543211	23
9	Naveen Singh	789 Market Lane, Delhi	9876543212	26
10	Aditya Patel	7898 Ocean Drive, Chennai	9876543214	27
11	Aman Chopra	246 Forest Avenue, Kolkata	9876543213	22
12	Avdeep Desai	34 River View, Hyderabad	9876543215	24



Topic :SQL DISTINCT Keyword

Example 1: Fetch Unique Names from the NAME Field.

Query:

```
SELECT DISTINCT NAME FROM students;
```

Output:

NAME
Shubham Kumar
Shreya Gupta
Naveen Singh
Aman Chopra
Aditya Patel
Avdeep Desai

output

The query returns only unique names, eliminating the duplicate entries from the table.



Topic :SQL DISTINCT Keyword

Example 2: Fetching Unique Combinations of Multiple Columns

If you want to retrieve unique combinations of NAME and AGE

Query:

```
SELECT DISTINCT NAME, AGE FROM students;
```

Output:

NAME	AGE
Shubham Kumar	23
Shreya Gupta	23
Naveen Singh	26
Aman Chopra	22
Aditya Patel	27
Avdeep Desai	24

THANK YOU